UPDATED PROJECT PROPOSAL

Pinboard Studio

AI image editing with Google Photos selection

PROJECT 2	REVISION 1.0	11 NOV 2026

Pivot decision: replace Pinterest as the source-photo provider with the Google Photos Picker API. Users search and choose their own photos inside Google Photos; Pinboard Studio imports the chosen original into its private Cloud Storage bucket for the requested edit. Google Drive is not used.

Executive summary

Pinboard Studio is a user-directed AI image-editing application. A user signs in to Pinboard Studio, authorizes access to Google Photos when they choose to import an image, selects a photo with Google’s secure Picker experience, and gives a text or voice instruction. The backend imports and validates the selected image, stores it privately, and sends it to the configured image-editing model only after the user understands and consents to that processing. The user reviews the result and can download it. Saving a generated result into Google Photos is an optional later capability, separate from selecting an existing photo.

Problem and objective

The original proposal depended on Pinterest OAuth, board/pin retrieval, and Pinterest publishing. Those permissions and approvals are a substantial external dependency. The revised project keeps the core educational and product goal—select a user-owned image, edit it with AI, and return a useful result—while using Google Photos Picker for the user's own library. The pivot does not claim that Google Photos is a public inspiration catalog or a Pinterest replacement.

Target user experience

1. The user signs into Pinboard Studio using Firebase Authentication.

2. When the user chooses **Choose from Google Photos**, the UI explains that a selected photo will be imported to Pinboard Studio storage for editing and may be sent to the named AI service. The user affirmatively continues.

3. The user connects/authorizes a Google account for Photos access. This Google OAuth grant is separate from the Firebase sign-in session.

4. The backend creates a Google Photos Picker session. The UI opens its pickerUri in a new tab/window; it must not embed the picker in an iframe.

5. The user searches in Google Photos and selects one or more items. The Picker API returns only items the user selected; it is not an API for silently searching the entire library.

6. Pinboard Studio shows the selected items and asks the user to choose one source image for the creation. The backend retrieves the selected bytes promptly while the temporary base URL is valid, validates the actual image, and stores the source in the private GCS bucket.

7. The user enters an editing prompt or records a voice note. Voice transcription is a separate workflow; the user reviews the resulting text before submitting.

8. With explicit consent for the visible editing feature, the backend sends the source image and final prompt to the selected Gemini image-editing service. Google Photos data must not be used to train or improve a general model.

9. The generated result is checked, stored in Cloud Storage, referenced in Firestore, and shown with a short-lived preview. The user may request one revision, then download the approved result.

10. When the user disconnects or requests deletion, remove OAuth credentials and related app-stored source/result data according to the product retention and deletion policy.

Google Photos integration boundaries

- **Picker API is the source-selection API.** The user searches and chooses photos in Google Photos. Pinboard Studio may present the selected items afterward, but cannot build an unrestricted Google Photos library browser from the Picker API.
- Picker media base URLs are temporary (documented as active for about 60 minutes and potentially shorter). Retrieve the selected media promptly; never treat a base URL as a durable image URL or persist it as the source reference.
- Google Photos APIs require user OAuth and do not support service-account authorization. Keep the OAuth client secret and user tokens on the backend, not in the React bundle.
- Use the minimum Picker scope for selection: `photospicker.mediaitems.readonly`. Do not request broad Photos-library scopes for this use case.
- Saving Pinboard Studio-generated images into the user's Google Photos library is a separate, optional feature. It requires the applicable Library API write scope and upload/batchCreate flow for app-created media; it does not mean the app can edit or overwrite the user's original Google Photos item.
- Pinterest board browsing and publishing are removed from the current product scope. Reconsider them only if a future version obtains Pinterest API access and the team deliberately reopens that scope.

System architecture

```
React / Vite UI (Firebase Hosting)
  ■■ Firebase Authentication: Pinboard Studio identity
  ■■ Google OAuth: separate Photos Picker authorization
  ■■ HTTPS REST API with Firebase ID token
       ■■ Node.js / Express API on Cloud Run
            ■■ verifies Firebase identity and creation ownership
            ■■ calls Google Photos Picker API as the user
            ■■ imports and validates selected image bytes
            ■■ Cloud Storage: private source/output/audio objects
            ■■ Firestore: profiles, creations, and workflow metadata
            ■■ Gemini: user-authorized image edit request
Generated output → private Cloud Storage → Firestore reference → UI preview/download
```

Data and storage

- Firebase Authentication remains the Pinboard Studio login and stable UID source.

- Firestore continues to store `users/{uid}` and `users/{uid}/creations/{creationId}`. Add only necessary source-provider metadata such as `sourceProvider`, media type, dimensions, and import timestamp.
- Store source and generated bytes in the existing private `project-2-pinboard-2026-media` bucket. Firestore records object paths and validated metadata, never image bytes, Google OAuth tokens in plaintext, or expiring Photos base URLs.
- Use a short-lived, owner-scoped import/session record with TTL. Delete staged and imported objects when the creation is cancelled, import fails, or the user requests deletion, subject to the published retention policy.

Privacy, consent, and compliance

- Explain in context what Google Photos data is requested, why the selected image is imported, where Pinboard Studio stores it, and which AI provider processes it. Require affirmative consent before access/transfer.
- Provide a privacy policy and user-facing instructions for disconnecting Google Photos and deleting app data. Honor deletion requests and secure all data in transit and at rest.
- Use Photos API data solely for visible, user-benefiting editing features. Do not sell it, use it for advertising, or use it to train/improve a general-purpose AI model. Obtain and document appropriate consent before permitted user-facing processing by another service, and verify that the AI provider terms satisfy Google’s Photos API policy.
- Complete applicable Google OAuth app verification before public launch. Confirm the current scope classification, consent-screen requirements, API quotas, and production access requirements in Google Cloud Console.

Scope

Included in revised core	Optional / later	Not in this pivot
Firebase sign-in; separate Google OAuth for Photos Picker; select/import one user-chosen photo; validate; private GCS source/result; Firestore metadata; text prompt; Gemini edit; result preview/download; deletion.	Pick multiple and choose within Pinboard; voice transcription; suggestions/review revision; save app-generated image into Google Photos with extra scope; advanced retention settings.	Pinterest OAuth, public Pinterest search, Pinterest board browser, Pinterest publishing, Google Drive storage, unrestricted Photos library search.

Current implementation status

Already present	Still required for this pivot
Firebase Authentication and UID-scoped API; Firestore profiles/creations; private GCS media upload/download lifecycle; 10-minute upload and 5-minute preview links; GCS staging cleanup; CI and UI scaffolding.	Google Photos OAuth/token lifecycle; Picker API session endpoints and UI; selected-item retrieval/import; actual image signature/dimension validation; Google Photos consent/privacy experience; real Gemini edit workflow and optional Photos export; production OAuth verification.

Success criteria

- An authenticated user can grant Photos access separately, launch the Google picker, search/select their own photo, and import that selection into the correct user-owned creation.
- The app never stores or relies on an expired Google Photos base URL; imported image bytes are validated and copied to private Cloud Storage.
- Unauthenticated or different users cannot access the creation, imported image, or generated output.
- The UI clearly explains any transfer to Gemini and asks for affirmative consent before processing Photos-derived content.
- The user can preview/download an edited result; cancellation, failure, disconnect, and deletion paths do not leave usable orphaned content or credentials.

- Automated tests cover OAuth state/ownership, picker polling, expiration, malformed/oversized images, import failures, cleanup, and API error handling; a live test uses an authorized tester account.

Risks and mitigations

Risk	Mitigation
OAuth verification/access delays	Enable the correct Photos Picker API early, configure test users and consent screen, complete applicable verification before public release.
Temporary picker/base URL expires	Follow returned pollingConfig, import immediately after selection, provide retry/reselect UX, and avoid storing baseUrl.
Photos data policy conflicts with AI processing/retention	Use a prominent in-context disclosure and affirmative consent, no model training, minimal retention, deletion controls, and verify AI-provider terms before enabling the edit call.
Large, malformed, or hostile image	Validate actual bytes and decoded dimensions, cap file/pixel sizes, reject unsupported encodings, strip unneeded metadata, and clean staging on failure.
Confusion between account identity and Photos permission	Keep Firebase Auth and Photos OAuth distinct in UI, backend session state, and documentation.

References

This revision is based on the submitted Pinboard Studio Project 2 proposal and current Google Photos Picker/Library API documentation. The requirements PDF that accompanies this proposal converts this architecture into implementation tasks, API/data contracts, security controls, and tests.

Official references

The API behavior and policy requirements below were checked against Google documentation on 11 November 2026. Re-check the current documentation before implementation and release.

Picker API: Get started — OAuth, create a session, open pickerUri, poll, list, retrieve.

Picker API: Sessions — Session lifecycle, pollingConfig, cleanup, and pickerUri behavior.

Picker API: Retrieve media items — Selected-item metadata and time-limited base URLs.

Google Photos API authorization scopes — Picker scope and app-created-data Library scopes.

Configure Google Photos APIs — Enable the correct API, configure OAuth client, web origins, and redirect URIs.

Photos API user-data policy — Appropriate use, disclosure, consent, data handling, and Limited Use rules.

Google Photos API UX guidelines — In-context disclosure, user control, and consent experience.

Google Photos API quotas — Current quotas and usage limits.